

Part 3

Module Reference

One chapter per source module - purpose, public surface, how the internals work function by function, assumptions, known drawbacks with line references, related tests, and a change guide.

AUDIENCE

Anyone editing or reviewing a specific file. Keep it open next to the code.

COVERS

`__init__`, `affine_extractor`, `sketch_library`, `sympy_tracer`, `z3_checker`, `lifter`, `emitter`, `cli`, `polygeist_frontend`, `mlir_validator`, `replay_checker`, `parser_corpus_compat`, `tests/fixtures`, `scripts`, `examples`

REPOSITORY STATE

branch main lineage, commit d1ab454

GENERATED

2026-09-20

SOURCE

`docs/handbook/source/03-module-reference.md`

Contents

How to Use This Reference	5
<code>__init__.py</code>	6
<code>affine_extractor.py</code> (Stage 1: parser)	7
Purpose	7
Public surface	7
Internals, in call order	7
Assumptions	8
Drawbacks	8
Tests	9
Change guide	9
<code>sketch_library.py</code> (target operation catalogue)	10
Purpose	10
The complete library	10
Drawbacks	12
Tests	12
Change guide	12
<code>sympy_tracer.py</code> (candidate scoring)	13
Purpose	13
Public surface	13
Internals	13
Drawbacks	14
Tests	14
Change guide	14
<code>z3_checker.py</code> (Stage 3: verifier)	15
Purpose	15
Public surface	15
Internals	15
Assumptions	16
Drawbacks	16
Tests	16
Change guide	16

lifter.py (orchestrator)	17
Purpose	17
Public surface	17
Internals	17
Drawbacks	17
Tests	18
Change guide	18
emitter.py (Stage 4: MLIR text generation)	19
Purpose	19
Public surface	19
Internals	19
Linalg handlers	20
StableHLO handlers	20
Drawbacks	21
Tests	21
Change guide	21
cli.py (command-line interface)	22
Purpose	22
Commands and options	22
Internals worth knowing	23
Drawbacks	23
Tests	23
Change guide	23
polygeist_frontend.py (cgeist wrapper)	24
Purpose	24
Internals	24
Drawbacks	24
Tests	24
mlir_validator.py (artifact validation)	25
Purpose	25
Internals	25
Drawbacks	25
Tests	25
replay_checker.py (counterexample triage, A-14)	26

Purpose	26
Internals	26
Drawbacks	26
Tests	26
parser_corpus_compat.py (corpus analysis, B-11)	27
Purpose	27
Internals	27
Drawbacks	27
tests/fixtures.py (kernel strings)	28
Purpose	28
Contents	28
Drawbacks	28
Scripts and Examples	29
scripts/generate_weekly_benchmark_report.py	29
scripts/generate_proof_quality_summary.py	29
scripts/analyze_parser_corpus_compat.py	29
scripts/run_batch_report_in_docker.ps1 and run_weekly_report_in_docker.ps1	29
examples/	29

How to Use This Reference

Every chapter has the same structure: **purpose**, **public surface**, **internals** (in the order a call flows through the file), **assumptions**, **drawbacks** (with severity and a pointer to the Part 5 finding), **tests** and a **change guide**. Line numbers refer to the code at the commit on the cover and will drift; search for the function name if they do.

File ownership follows the team's lanes: Lane A owns `z3_checker.py`; Lane B owns `affine_extractor.py`, `emitter.py` and `sympy_tracer.py`; Lane C owns `cli.py`, `mlir_validator.py` and the tests; `lifter.py` is shared and changes to it must be announced. Modules not listed (`sketch_library.py`, `polygeist_frontend.py`, `replay_checker.py`, `parser_corpus_compat.py`, scripts) should be assigned explicitly; this reference suggests an owner for each.

Module	Lines	Suggested owner	Risk level
<code>z3_checker.py</code>	1,736	A	Critical
<code>cli.py</code>	1,520	C	Medium
<code>emitter.py</code>	1,430	B	High
<code>affine_extractor.py</code>	1,301	B	High
<code>sketch_library.py</code>	752	A (semantics) with B review (emitter impact)	Medium
<code>lifter.py</code>	690	shared	High
<code>sympy_tracer.py</code>	485	B	Medium
<code>polygeist_frontend.py</code>	446	C	Low
<code>replay_checker.py</code>	282	A	Low
<code>mlir_validator.py</code>	247	C	High
<code>parser_corpus_compat.py</code>	100	B	Low
<code>tests/fixtures.py</code>	792	C	Medium
<code>__init__.py</code>	45	C	Low

`__init__.py`

Purpose. Package entry point. Its docstring lists seven sub-modules (it omits the frontend, validator, replay checker and corpus tool).

Public surface. Re-exports `AffineExtractor`, `LoopNestInfo`, `SKETCH_LIBRARY`, `OperationSketch`, `Z3EquivalenceChecker`, `CheckResult`, `SympyTracer`, `LinalgEmitter`, `StableHLOEmitter`, `Lifter`, `LiftResult`, `LiftResultState`, `PolygeistFrontend`, `PolygeistFrontendError`, `CgeistInvocation`, `CgeistResult`; defines `__version__ = "0.1.0"`.

Drawbacks.

Severity	Issue
Low	Importing the package imports Z3 and SymPy eagerly (about a second), even for <code>loophole --help</code> or <code>compile</code> .
Low	Version duplicated in <code>pyproject.toml</code> , <code>cli.py</code> and the emitter header (F-25).
Low	Mixed indentation (two and four spaces) in <code>__all__</code> imports, harmless but a sign of unreviewed edits.

Change guide. Keep the public surface small; anything imported by scripts or tests from a private name (underscore) should either become public here or stop being imported.

affine_extractor.py (Stage 1: parser)

Purpose

Turn textual MLIR containing an affine/SCF loop nest into a `LoopNestInfo`. It is a hand-written line scanner: no MLIR library is used (the optional `mlir.ir` syntax check only runs if MLIR Python bindings happen to be installed, which they are not in any supported setup).

Public surface

- Data classes `AccessPattern`, `ComputeOp`, `ParsingWarning`, `LoopNestInfo` (described in Part 2).
- `AffineExtractor(validate_with_mlir=True, verbose_diagnostics=False)` with `extract(mlir_text)` -> `LoopNestInfo` and a `diagnostics` list reset per call.

Internals, in call order

Module helpers (106-330).

- `_split_top_level(text, delimiter)`: splits on commas outside `()`, `[]`, `<>`.
- `_strip_inline_comment, _find_matching(text, open_idx, open, close)`: bracket matching.
- `_replace_token_boundary(expr, old, new)`: replaces a token only when surrounded by non-identifier characters (identifier characters include `%`).
- `_strip_ssa_percent: %foo to foo.`
- `_extract_memref_payload, _parse_memref_type: memref<4x?xf32> to ([4, -1], "f32")`; `*` or unranked gives `[]`; `?`, `-1` and `*` dims become `-1`.
- `_apply_known_constants(expr, const_map)`: substitutes constant values for both `%c1` and `c1` (the `%`-less replacement is the numbered-constant bug, F-04).
- `_canonicalize_index_expr`: SymPy `expand` plus lexicographic printing, so equivalent affine forms compare equal as strings; falls back to whitespace normalisation.
- `_normalize_index(idx, iv_map, const_map, apply_map)`: in order, substitutes `affine.apply` results (parenthesised), renames IV SSA names to short names, substitutes constants, strips `%`, canonicalises.

`extract` (345-390). Normalises newlines, resets diagnostics, optional MLIR validation, then:

1. `_parse_func_declaration`: first `func.func @`, name up to `(`, arguments via bracket matching and top-level comma split into `name: type`.
2. `_extract_constants`: every line with `=` and `arith.constant`; integer or float literal; `dense<...>` and unparseable literals produce warnings.
3. `_parse_loop_structure`: every line containing `affine.for` (anywhere in the line, so `%0 = affine.for ... iter_args` is accepted) or starting with `scf.for`. Each gives `(iv_ssa, lb, ub)`; `step` and `iter_args` suffixes are cut off and discarded. IV short names come from `_allocate_iv_name` (strip `%`, prefix `iv<N>` if not alphabetic, de-duplicate with `_1`). Lines that look like loops but do not parse give debug diagnostics.
4. `_extract_affine_apply_map`: parses `%x = affine.apply affine_map<(d0, ...)[s0, ...] -> (expr)>(%ops)` into `{ "%x": "canonical expr over IV short names" }`. Symbol operands are ignored.
5. `_collect_accesses`: `affine.load/memref.load (%r = ... %T[idx, ...] : type)` and `affine.store/memref.store (%v, %T[idx, ...] : type)`, each via `_parse_load` or `_parse_store`, with index normalisation.

6. `_collect_compute_ops`: every line containing `= arith.;` `constant` gets a float value (0.0 on failure, with warning); unary `negf`, `extf`, `truncf`; others keep the first two operands (warning if more).
7. `_add_unsupported_form_diagnostics`: warnings for `scf.for` with `iter_args`, any `affine.if/scf.if`, no loops, no writes.
8. Bounds resolved with `_resolve_bound`: integer literal, known constant, or the raw string with `%` stripped.
9. `_resolve_tensor_metadata`: collects shapes from argument types and every access type, ranks from index arity; `_normalize_tensor_shape_metadata` picks the majority rank and, per dimension, the unique static value or -1 if conflicting (B-15, B-16); `_normalize_tensor_element_type` picks the dominant type.
10. `_dominant_element_type`: most common type, `f32` if none (warning).
11. `_classify_iv_roles`: an IV is parallel if its name is a substring of any write index expression.
12. `_detect_accumulation`: output tensor also read and the stored value produced by `addf/addi`; otherwise any add op counts (debug diagnostic).

Assumptions

- One operation per line, and each operation fully on one line.
- One function per input (the whole text is scanned regardless of how many functions it contains).
- Loops are perfectly nested and every access belongs to the innermost body.
- The first store is the output; there is one output tensor.
- Unit loop step; loop bounds are literals, constants or simple symbols.

Drawbacks

Severity	Issue	Where	Finding
Critical	Loop <code>step</code> discarded	637-638, 662-663	F-03
High	IV role classification by substring	1234-1239	F-04
High	<code>%</code> -stripped constant names replace integer literals	265-279	F-04
High	Multi-function modules merged	345-390	F-05
Medium	<code>min/max</code> affine bounds kept as raw strings	1279-1301	F-16
Medium	Any add treated as accumulation	1265-1275	F-20
Medium	Silent defaults: operand truncation, constant 0.0, type <code>f32</code>	1054-1093, 1207-1217	F-21
Medium	<code>iter_args/affine.yield</code> not modelled; works for matmul only by accident	619-647	F-21
Low	Docstring claims no regex; module uses <code>re</code>	1-14, 297	F-22
Low	Optional MLIR validation never runs in supported setups	16-19, 451-469	F-22
Low	<code>loop_order</code> is a copy of <code>induction_vars</code> ; loop nesting not recorded	669-718	n/a

Tests

`tests/unit/test_affine_extractor.py` (49 tests: per-kernel parsing, mixed affine/SCF, index normalisation, stress fixtures B-10, dynamic memref B-15), `test_parser_diagnostics.py` (11), `test_parser_unsupported_form_diagnostics.py` (4), `test_parser_corpus_compat.py` (3), `test_fixture_source_policy.py` (2). There are no negative tests showing that a malformed or unusual kernel does *not* parse into something plausible.

Change guide

- Adding a new construct: add a field to `LoopNestInfo` rather than encoding it in strings, add a fixture to `fixtures.py`, add a parser test, and decide how `z3_checker` must use the field (or make `structural_match` reject kernels that use it until supported).
- Replacing substring checks: reuse `_replace_token_boundary`'s boundary rule or tokenise with a regex `\b` match.
- Longer term, consider splitting the text into function bodies and regions using `_find_matching` before any scanning, or moving to MLIR Python bindings (Part 5, F-34).

sketch_library.py (target operation catalogue)

Purpose

Declares every target operation LoopHole can produce, in a loop-level form the verifier can reason about. It contains no logic beyond derived properties.

The complete library

Order matters: `SKETCH_LIBRARY` is iterated in this order, and ties in candidate ranking are broken by it. "P/R" is the number of parallel and reduction dimensions. "mlir-opt 17" is the result of validating the emitted artifact for a fixture that reaches this handler (Part 5 chapter 3); "not reached" means no fixture currently selects this sketch.

#	Name	Dialect	Dims (P/R)	Indexing maps (inputs..., output)	Payload	mlir-opt 17
1	<code>linalg.conv_2d_nhwc_hwc</code>	<code>linalg</code>	<code>n,oh,ow,kh,kw,ic,oc (4/3)</code>	<code>(n, oh+kh, ow+kw, ic); (kh, kw, ic, oc); (n, oh, ow, oc)</code>	<code>multiply_accumulate</code>	OK
2	<code>linalg.conv_2d_nchw_fchw</code>	<code>linalg</code>	<code>n,oc,oh,ow,ic,kh,kw (4/3)</code>	<code>(n, ic, oh+kh, ow+kw); (oc, ic, kh, kw); (n, oc, oh, ow)</code>	<code>multiply_accumulate</code>	not reached
3	<code>linalg.conv_2d</code>	<code>linalg</code>	<code>oh,ow,kh,kw (2/2)</code>	<code>(oh+kh, ow+kw); (kh, kw); (oh, ow)</code>	<code>multiply_accumulate</code>	OK
4	<code>linalg.conv_1d_nwc_wcf</code>	<code>linalg</code>	<code>n,w,kw,c,f (3/2)</code>	<code>(n, w+kw, c); (kw, c, f); (n, w, f)</code>	<code>multiply_accumulate</code>	not reached
5	<code>linalg.conv_1d_ncw_fcw</code>	<code>linalg</code>	<code>n,w,kw (2/1)</code>	<code>(n, w+kw); (kw); (n, w)</code>	<code>multiply_accumulate</code>	FAIL rank
6	<code>linalg.pooling_nhwc_max</code>	<code>linalg</code>	<code>n,oh,ow,kh,kw,c (4/2)</code>	<code>(n, oh+kh, ow+kw, c); (kh, kw); (n, oh, ow, c)</code>	<code>accumulate_max</code>	not reached
7	<code>linalg.batch_matmul</code>	<code>linalg</code>	<code>b,m,n,k (3/1)</code>	<code>(b, m, k); (b, k, n); (b, m, n)</code>	<code>multiply_accumulate</code>	not reached
8	<code>linalg.matmul</code>	<code>linalg</code>	<code>m,n,k (2/1)</code>	<code>(m, k); (k, n); (m, n)</code>	<code>multiply_accumulate</code>	OK
9	<code>linalg.matvec</code>	<code>linalg</code>	<code>m,k (1/1)</code>	<code>(m, k); (k); (m)</code>	<code>multiply_accumulate</code>	OK
10	<code>linalg.vecmat</code>	<code>linalg</code>	<code>k,n (1/1)</code>	<code>(k); (k, n); (n)</code>	<code>multiply_accumulate</code>	not reached
11	<code>linalg.dot</code>	<code>linalg</code>	<code>k (0/1)</code>	<code>(k); (k); ()</code>	<code>multiply_accumulate</code>	FAIL rank
12	<code>linalg.reduce{arith.addf}_rowsum</code>	<code>linalg</code>	<code>i,j (1/1)</code>	<code>(i, j); (i)</code>	<code>accumulate_add</code>	FAIL syntax
13	<code>linalg.reduce{arith.addf}_colsum</code>	<code>linalg</code>	<code>i,j (1/1)</code>	<code>(i, j); (j)</code>	<code>accumulate_add</code>	FAIL syntax
14	<code>linalg.reduce{arith.maxf}</code>	<code>linalg</code>	<code>i,j (1/1)</code>	<code>(i, j); (i)</code>	<code>accumulate_max</code>	FAIL syntax
15	<code>linalg.map{arith.maxf_zero}</code>	<code>linalg</code>	<code>i,j (2/0)</code>	<code>(i, j); (i, j)</code>	<code>relu</code>	reached by <code>relu</code> since LOOPH-20; not yet validated (expected FAIL syntax, F-08)

#	Name	Dialect	Dims (P/R)	Indexing maps (inputs..., output)	Payload	mlir-opt 17
16	linalg.map{arith.addf}	linalg	i,j (2/0)	(i, j) x3	add	FAIL syntax
17	linalg.map{arith.mulf}	linalg	i,j (2/0)	(i, j) x3	multiply	reached by <code>elementwise_mul</code> since LOOPH-20; not yet validated (expected FAIL syntax, F-08)
18	linalg.map{arith.subf}	linalg	i,j (2/0)	(i, j) x3	subtract	FAIL syntax
19	linalg.map{arith.addf}_1d	linalg	i (1/0)	(i) x3	add	not reached
20	linalg.map{arith.mulf_scalar}	linalg	i,j (2/0)	(i, j); (i, j)	scale	FAIL syntax; no fixture reaches it since LOOPH-20
21	linalg.transpose	linalg	i,j (2/0)	(i, j); (j, i)	copy	OK
22	linalg.copy	linalg	i,j (2/0)	(i, j); (i, j)	copy	OK
23	stablehlo.dot_general	stablehlo	m,n,k (2/1)	as matmul	multiply_accumulate	cannot validate
24	stablehlo.dot_general_matvec	stablehlo	m,k (1/1)	as matvec	multiply_accumulate	cannot validate
25	stablehlo.dot_general_vecdot	stablehlo	k (0/1)	as dot	multiply_accumulate	cannot validate
26	stablehlo.transpose	stablehlo	i,j (2/0)	(i, j); (j, i)	copy	cannot validate
27	stablehlo.add	stablehlo	i,j (2/0)	(i, j) x3	add	cannot validate
28	stablehlo.subtract	stablehlo	i,j (2/0)	(i, j) x3	subtract	cannot validate
29	stablehlo.multiply	stablehlo	i,j (2/0)	(i, j) x3	multiply	cannot validate
30	stablehlo.reduce{add}	stablehlo	i,j (1/1)	(i, j); (i)	accumulate_add	cannot validate
31	stablehlo.reduce{add}_colsum	stablehlo	i,j (1/1)	(i, j); (j)	accumulate_add	cannot validate
32	stablehlo.reduce{max}	stablehlo	i,j (1/1)	(i, j); (i)	accumulate_max	cannot validate
33	stablehlo.convolution_1d	stablehlo	n,w,kw (2/1)	as conv_1d_ncw_fcw	multiply_accumulate	cannot validate
34	stablehlo.convolution_2d	stablehlo	oh,ow,kh,kw (2/2)	as conv_2d	multiply_accumulate	cannot validate
35	stablehlo.convolution	stablehlo	as conv_2d_nhwc_hwc (4/3)	as conv_2d_nhwc_hwc	multiply_accumulate	cannot validate

Drawbacks

Severity	Issue	Finding
High	Payload labels are too coarse to express real semantics (for example scale's constant, relu's zero, pooling's window operand), and the verifier trusts them for the source side	F-01, F-18
Medium	Several sketches are indistinguishable structurally (row sum vs row max apart from payload; transpose vs copy apart from maps; conv_1d_ncw_fcw vs matmul by counts), making ranking decide more than it should	F-19
Medium	Pooling sketch invents a window operand that no source kernel reads	n/a
Low	Module docstring describes a matching pipeline (num_reads/num_writes checks) that does not exist	F-25
Low	<code>IteratorType.WINDOW</code> unused	F-25
Low	No 3-D transpose sketch although the emitter supports arbitrary permutations (the 3-D fixtures always fail to lift)	n/a

Tests

`test_sketch_library.py` (22 tests) checks presence, uniqueness, field shapes, and the matmul sketch's dimensions. `test_emitter.py::test_stablehlo_handler_map_covers_all_sketches` checks the StableHLO handler table.

Change guide

Adding a sketch requires, in one PR: the sketch; an emitter handler; a correct fixture; at least one mutant fixture that must not match; a real-toolchain artifact validation test; and a numerical oracle reference implementation once T11 exists. Place it in the list before less specific sketches with the same counts.

sympy_tracer.py (candidate scoring)

Purpose

Produce a symbolic expression of the loop body and a heuristic confidence that a sketch fits. It also contains a convolution pattern recogniser. Nothing in this module affects correctness directly, but ranking decides which candidates Z3 sees first, and the first proof wins.

Public surface

`TraceResult`; `SympyTracer` with `trace(loop)`, `infer_sketch(result, loop, target_dialect)` (used only by `examples/run_demo.py`), `recognise_convolution(loop)`, and the de-facto public `_sketch_confidence(result, loop, sketch)` called by the lifter.

Internals

- `trace`: creates integer, non-negative symbols per IV and an `IndexedBase` per tensor; `_build_body_expr` maps each read's SSA name to `T[idx]`, walks compute ops in order via `_eval_compute_op` (mul, add, sub, div, max, min, neg, constant), takes the value of the first store, subtracts the old output read if accumulating, then wraps with `Sum` over each reduction IV and calls `doit()` and `expand`.
- `_sympy_index_expr`: converts index strings to SymPy, using a placeholder-replacement dance that effectively reduces to `sympify(expr, locals=iv_syms)` with a manual fallback.
- `_sketch_confidence`: weighted score normalised by 12.

Criterion	Weight	Test
Loop count equals sketch loops	2	<code>len(induction_vars) == num_loops</code>
Reduction count equals	2	<code>len(reduction_vars) == num_reduction</code>
Payload match	3 (1.5 if "compatible")	<code>_infer_cp(loop)</code> vs sketch payload; compatible pairs are <code>multiply_accumulate/accumulate_add</code> , <code>add/accumulate_add</code> , <code>copy/scale</code>
Distinct input tensor count	1	<code>equals num_inputs</code>
Algebraic structure	2 x score	<code>_algebraic_structure_matches</code> : products-and-sums for <code>multiply_accumulate</code> , single indexed symbol for copy, <code>Add</code> node for add payloads, <code>Max</code> for max/relu, 0.5 otherwise
Access pattern	2	reduction IVs absent from write indices and present in some input index (substring tests)

- `recognise_convolution`: if some non-output read index contains `+` and a reduction IV name as a substring, return `linalg.conv_1d_ncw_fcw` (1 reduction, at most 3 loops), `linalg.conv_2d` (2 reductions, 4 loops) or `linalg.conv_2d_nhwf_hwc` (at least 2 reductions, at least 6 loops).

Drawbacks

Severity	Issue	Finding
Medium	Confidence ignores which indices appear where; matmul and conv_1d both score 1.00 on the matmul fixture	F-19
Medium	<code>_infer_cp</code> disagrees with the verifier's <code>_infer_compute_payload</code> (mul+add without reduction: ADD here, MULTIPLY there)	F-23
Medium	Substring IV checks in access pattern and convolution recogniser	F-04
Medium	Recogniser only returns Linalg names, so it never helps StableHLO, and its match overrides confidence in the lifter's sort	F-19
Low	<code>trace</code> is expensive for large bounds (symbolic sums over concrete ranges expand to hundreds of terms); failures are swallowed and every candidate gets confidence 0	n/a
Low	Many unused imports; <code>infer_sketch</code> unused by the product	F-25

Tests

No dedicated unit test file. Covered indirectly by lifter and integration tests, and by `test_dot.py::test_dot_proof_has_no_sympy_z3_disagreement`.

Change guide

If ranking matters after T5 makes proofs sound, add a unit test file with explicit expected rankings, and make access-pattern scoring compare index structure (for example via the same dimension alignment the verifier uses). Otherwise consider verifying all structural candidates and dropping the confidence score to a tie-breaker.

z3_checker.py (Stage 3: verifier)

Purpose

Decide whether a loop nest and a sketch compute the same output values, using Z3. It is the largest and most important module and the one with the most serious defects.

Public surface

- `CheckResult`, `ReductionPatternKind`, `VerificationReport`.
- `structural_match(loop, sketch) -> bool`.
- `Z3EquivalenceChecker(timeout_ms=10000, unroll_threshold=32)` with `check(loop, sketch)`.
- Used from outside although private: `_verify_parametric` (lifter), `_collect_shape_dims` and `_classify_reduction_pattern` (tests).

Internals

Helpers (104-287). `_make_tensor_func` (uninterpreted $\text{Int}^{\text{rank}} \rightarrow \text{Real}$), `_z3_int`, `_sanitize_symbol_name`, `_tokenize_affine_expr` and `_AffineExprParser` (recursive descent for `+ - * ( )` over integers and names), `_eval_affine_expr`, `_eval_index_expr` (falls back to a fresh integer symbol named after the whole expression if parsing fails, which silently makes unknown indices unconstrained).

`structural_match` (396-459). Requires equal loop count and reduction count and at least one write; checks op-type presence for `multiply_accumulate`, `copy` (no mul/add), `add`, `subtract`, `multiply`, `accumulate_add`, `accumulate_max`, `relu`, `negate`, `max` and `min`. Scale needs a multiply with exactly one loaded operand (`_multiplies_one_loaded_value`).

`check` (467-499). Pre-filter, `_verify`, catches any exception as `ENCODE_ERROR` with the message in notes, records elapsed time.

`_verify_concrete` (524-602). Builds tensor functions, IV integers, source and sketch assertions and optional domain assumptions; checks both implications on a fresh solver state with `timeout` set; converts results via `_failed_implication_report` (extracts the model and bindings when `sat`).

Source side. `_build_source_assertions` (714-775), `_build_rhs_expr` (777-802), `_build_elementwise_expr` (804-845), `_build_reduction_expr` (847-893, dispatching to the strategies in Part 2), `_unrolled_reduction` (1109-1168), `_guarded_symbolic_unroll_reduction` (1023-1056), `_guarded_symbolic_unroll_reduction_multi` (1058-1107), `_body_term_at` (1170-1199), `_recfunc_reduction` (1201-1260).

Sketch side. `_build_sketch_assertions` (1266-1336) aligns dimensions (`_align_dims`, 1602-1625), binds sketch operand i to the i -th unique non-output read tensor, evaluates maps with `_eval_sketch_map` (1627-1642), maps a scalar `()` output to index `[0]` when the output is `memref<1xT>`; `_build_sketch_rhs` (1338-1546) mirrors every reduction strategy for the sketch side; `_read_at` (1548-1560).

Reduction classification and bounds. `_classify_reduction_pattern` (895-935), `_symbolic_reduction_upper_bound_multi` (937-995), `_symbolic_reduction_upper_bound` (997-1021), `_build_symbolic_shape_assumptions` (657-708), `_parse_simple_iv_offset` (616-655), `_bound_expr` (372-396), `_bound_as_int` (398-405).

Payload inference. `_infer_compute_payload` (1779-1794) looks up the exact op-kind set in `_PAYLOAD_BY_OP_KINDS`; `_is_max_with_zero` (374-389) separates RELU from MAX and refuses a zero that

came from an unparseable literal. Anything unmodelled raises `_UnmodelledComputePayload` (`ENCODE_ERROR`).

Parametric pilot. `_collect_shape_dims` (1676-1693), `_verify_parametric` (1695-1736).

Assumptions

- Tensors are total functions; out-of-bounds reads are not modelled.
- Source semantics are fully captured by (payload label, the one or two non-output reads, index expressions). Any other read count is `ENCODE_ERROR`.
- Values are mathematical reals.
- Parallel IVs of source and sketch correspond in order.
- Every loop visits every integer in `[lo, hi)`.

Drawbacks

Severity	Issue	Where	Finding
Critical	Source term built from payload category, not dataflow	841-976, 1332-1391	F-01
High	Output initial value ignored; <code>-1e30</code> max start	756-773, seven sites	F-07
Medium	Parametric pilot collapses equal extents, bounded by static shapes	1676-1736	F-14
Medium	Unparseable index becomes an unconstrained symbol instead of an error	282-286	F-21
Medium	Reduction strategy logic duplicated for source and sketch sides (about 350 lines twice)	847-1260, 1378-1546	F-23
Low	<code>_verify_symbolic</code> is a wrapper around the concrete path; module docstring misleading	1566-1596	F-25
Low	Instance state <code>_bound_symbols</code> makes the checker non-reentrant	370, 477	n/a
Low	Unused imports (<code>Tactic</code> , <code>Then</code> , <code>Sum</code> , <code>Array</code> , <code>Store</code> , <code>Select</code> , ...) and unused locals	33-39	F-25

Tests

`test_z3_checker.py` (13), `test_a13_multi_reduction_boundary.py` (16), `test_a15_shape_parametric_proof.py` (26), plus most integration tests. None checks that a wrong kernel is *not* proved (T1 adds these).

Change guide

- Do not change the encoding without the T1 mutant tests present; they are the only way to see soundness regressions.
- T5's design: build Z3 terms by walking backwards from the stored SSA value through `compute_ops`, mapping loads to tensor applications, constants to `RealVal`, and each arith op to the corresponding Z3 operator in operand order; raise on anything unmodelled.
- When adding a reduction strategy, write it once as a function of "how to read operand *i* at a point" and use it for both sides.
- Z3 `RecFunction` names are process-global: name every new one with `_fresh_rec_name`.
- Apply sketch operand functions only after `_check_operand_rank`; it turns a map/tensor rank mismatch into a named `ENCODE_ERROR` instead of a raw Z3 arity error.

lifter.py (orchestrator)

Purpose

Run stages 1 to 4, choose a result, and define the result types and policy profiles shared by the CLI, scripts and tests.

Public surface

`PolicyProfile`, `POLICY_PROFILES`, `POLICY_PROFILE_ALIASES`, `POLICY_PROFILE_CHOICES`, `POLICY_PROFILE_ENV_VAR`, `DEFAULT_POLICY_PROFILE`, `resolve_policy_profile`; `LiftResultState`, `LiftResult`; `Lifter(target, z3_timeout_ms, top_k, strict_mode, verbose, parametric_mode)` with `from_policy_profile(...)`, `lift(mlir_text)`, `lift_many(texts)`; module function `lift(mlir_text, target, z3_timeout_ms, strict_mode, verbose)`.

Internals

- `_build_strided_conv_sketch` (49-131): for convolution sketches, rewrites the input indexing map with coefficients inferred from the source index expressions ($2*w + 3*kw$), returning a modified copy, or `None` when all coefficients are 1.
- `lift` (380-529): parse (exceptions become "Parse error"), match, verify, optional parametric augmentation, then the decision chain described in Part 2 (refuted, strict rejection, emission, success).
- `_match_candidates` (539-592): chooses the library by target (`both` uses all 35), runs the convolution recogniser and SymPy trace (trace failures give confidence 0), keeps structurally matching sketches, sorts by (`recogniser match`, `confidence`) descending, returns $2 * top_k$.
- `_annotate_disagreement` (594-614): marks high-confidence unproved or refuted results.
- `_verify_candidates` (620-675): see Part 2.
- `_emit` (681-685): StableHLO emitter when the sketch's dialect is StableHLO or the target is StableHLO, else Linalg.

Drawbacks

Severity	Issue	Finding
Medium	<code>result_state</code> maps no-verification to REFUTED; no-verdict reports hidden whenever another candidate has a result	F-12
Medium	First EQUIVALENT wins; recogniser overrides confidence; $2*top_k$ returned but <code>top_k</code> verified	F-19
Medium	Emission error after a proof still yields state PROVED	F-12
Low	Imports private <code>_infer_linear_coeff</code> from <code>emitter.py</code>	F-23
Low	<code>lift_many</code> unused; module-level <code>lift</code> does not accept profile or parametric flags	F-25
Low	Section comment "Result type" sits above the strided sketch builder and profiles, reflecting unplanned growth	n/a

Tests

`test_lifter_semantics.py` (9) for states and profiles; `test_a15_*` for parametric mode; integration tests use `lift` and `Lifter`.

Change guide

Coordinate every change in the lane channels (T4, T8 and T10 all touch this file). New result states must be reflected in `cli.py` colours and exit codes, the batch report schema, the weekly scripts and `LiftResult.summary`.

emitter.py (Stage 4: MLIR text generation)

Purpose

Produce MLIR text for a matched sketch using shapes and names from `LoopNestInfo`.

Public surface

`EmissionError`; `LinalgEmitter().emit(sketch, loop, func_name=None)`; `StableHLOEmitter().emit(sketch, loop, func_name=None)`; helper `_infer_linear_coeff` (used by the lifter).

Internals

- Type helpers `_memref_type`, `_tensor_type`, `_map_elem_type` (maps C type names that never occur).
- Metadata gates: `_require_element_type`, `_require_output_tensor`, `_require_input_tensor`, `_require_shape` (optionally rank), `_require_iv`, `_find_read_access_expr`.
- Convolution attribute inference: `_infer_linear_coeff(expr, var)` (SymPy `coeff`, or regex parsing of `N*var`, `var*N` and signed terms when SymPy is missing), `_infer_stride_dilation_from_expr`, `_infer_conv2d_window_attrs` (finds input dimensions depending on exactly one parallel and one reduction IV).
- `_bound_size`: loop range size, treating non-integer bounds as 0 to 1.

Linalg handlers

Sketch	Method	Output shape and notes	mlir-opt 17
<code>linalg.matmul</code>	<code>_emit_matmul</code>	ranks 2,2,2	OK
<code>linalg.transpose</code>	<code>_emit_transpose</code>	permutation from write index, validated against shapes	OK
<code>linalg.dot</code>	<code>_emit_dot</code>	output rank 1 (<code>memref<1xT></code>)	FAIL: needs rank-0 output
<code>linalg.matvec</code>	<code>_emit_matvec</code>	ranks 2,1,1	OK
<code>linalg.vecmat</code>	<code>_emit_vecmat</code>	ranks 1,2,1	not reached
<code>linalg.batch_matmul</code>	<code>_emit_batch_matmul</code>	ranks 3	not reached
<code>linalg.conv_1d_ncw_fcw</code>	<code>_emit_conv1d_ncw</code>	ranks 2,1,2 with inferred stride/dilation	FAIL: op needs rank 3
<code>linalg.conv_1d_nwc_wcf</code>	<code>_emit_conv1d_nwc</code>	ranks 3	not reached
<code>linalg.conv_2d</code>	<code>_emit_conv2d_simple</code>	ranks 2	OK
<code>linalg.conv_2d_nhwc_hwc</code>	<code>_emit_conv2d_nhwc</code>	ranks 4	OK
<code>linalg.conv_2d_nchw_fchw</code>	<code>_emit_conv2d_nchw</code>	ranks 4; unused defaults <code>KH=3</code> , <code>KW=3</code> computed	not reached
<code>linalg.pooling_nhwc_max</code>	<code>_emit_pool2d_max</code>	invents <code>%W</code> window operand; unit strides	not reached
<code>linalg.copy</code>	<code>_emit_copy</code>	<code>linalg.copy</code>	OK
<code>linalg.map{arith.addf}, _1d, mulf, subf</code>	<code>_emit_elementwise</code>	region syntax (<code>{%a: T, %b: T} -> {...}</code>)	FAIL: syntax
<code>linalg.map{arith.maxf_zero}</code>	<code>_emit_relu</code>	same syntax, <code>arith.maxf</code>	expected FAIL
<code>linalg.map{arith.mulf_scalar}</code>	<code>_emit_scale</code>	adds free <code>%alpha</code> argument	FAIL: syntax
<code>linalg.reduce{...} rowsum, colsum</code>	<code>_emit_reduce_sum</code>	<code>dimensions = [1]</code> or <code>[0]</code> by name	FAIL: syntax
<code>linalg.reduce{arith.maxf}</code>	<code>_emit_reduce_max</code>	fixed <code>dimensions = [1]</code>	FAIL: syntax

StableHLO handlers

Sketches	Method	Notes
<code>dot_general</code> , <code>_matvec</code> , <code>_vecdot</code>	<code>_emit_dot_general</code>	<code>contracting_dims</code> by rank combination; returns fresh tensor
<code>transpose</code>	<code>_emit_transpose</code>	duplicate of Linalg permutation logic
<code>add</code> , <code>subtract</code> , <code>multiply</code>	<code>_emit_elementwise</code>	a RELU branch exists but no RELU StableHLO sketch reaches it
<code>reduce{add}</code> , <code>_colsum</code> , <code>reduce{max}</code>	<code>_emit_reduce</code>	dims from reduction IV positions; max init <code>-3.4e38</code> or <code>-1.8e308</code>
<code>convolution_1d</code> , <code>_2d</code> , <code>convolution</code>	<code>_emit_convolution</code>	reshapes operands to rank 4, <code>dim_numbers = [b, 0, 1, f]x[0, 1, i, o]->[b, 0, 1, f]</code>

Drawbacks

Severity	Issue	Finding
High	Invalid templates for map, reduce, dot, conv_1d	F-08
High	Whole-tensor ops emitted for partial loops; free <code>%alpha</code>	F-08 (T8)
High	StableHLO drops the output's initial value	F-07
Medium	Transpose and metadata logic duplicated between the two classes	F-23
Medium	1-D StableHLO convolution reshapes to a 2-D window with a unit dimension; unverified	F-09
Low	Unreachable <code>_emit_generic</code> ; unused computed sizes; <code>_map_elem_type</code> no-op	F-25

Tests

`test_emitter.py` (42 tests, mostly string assertions; artifact validation only for matmul, transpose, conv2d),
`test_stablehlo_phase3_ops.py` (10), conv, transpose and matmul integration tests.

Change guide

Every handler change must come with an artifact-validation test run against the real tool for that dialect. Prefer generating text from a single small builder per dialect (function signature, `ins/outs`, region) over per-op f-strings.

cli.py (command-line interface)

Purpose

User entry point, built with Click (commands) and Rich (panels, tables, syntax highlighting, progress spinners). Installed as the `loophole` console script and also runnable with `python -m loophole.cli`.

Commands and options

Command	Arguments and options (defaults)	What it does	Exit codes
<code>compile</code>	<code>SOURCE_FILE</code> ; <code>-o</code> (source stem <code>.mlir</code>); <code>--docker-image</code> (GHCR name); <code>--std</code> ; <code>--frontend-binary</code> ; <code>--no-affine-raise</code> ; <code>--print-command</code>	Docker `cgeist ...	sed
<code>cgeist</code>	<code>SOURCE_FILES...</code> ; <code>-o</code> ; <code>-x c</code>	cpp (<code>c</code>); <code>--function</code> ; <code>-l</code> ; <code>-D</code> ; <code>--std</code> ; <code>--clang-arg</code> ; <code>--cgeist-bin</code> (<code>cgeist</code>); <code>--docker-image</code> ; <code>--timeout-sec</code> (<code>60</code>); <code>-v`</code>	<code>PolygeistFrontend</code> with Docker preferred; prints or writes MLIR
<code>lift</code>	<code>INPUT_FILE</code> ; <code>-o</code> ; <code>-t linalg</code>	stablehlo	both (<code>linalg</code>); <code>--profile</code> (<code>default</code>); <code>--strict</code> ; <code>--z3-timeout</code> (<code>10000</code>); <code>--top-k</code> (<code>3</code>); <code>-v</code> ; <code>--report</code> ; <code>--no-verify</code> ; <code>--parametric</code> ; <code>--show-replay`</code>
<code>lift-c</code>	<code>SOURCE_FILES...</code> ; <code>-o</code> ; <code>--mlir-output</code> ; <code>-t</code> ; <code>--profile</code> ; <code>--strict</code> ; <code>--z3-timeout</code> (<code>10000</code>); <code>--top-k</code> ; <code>--no-verify</code> ; <code>--report</code> ; <code>-x</code> ; <code>--function</code> ; <code>-I</code> ; <code>-D</code> ; <code>--std</code> ; <code>--clang-arg</code> ; <code>--cgeist-bin</code> ; <code>--docker-image</code> ; <code>--timeout-sec</code> ; <code>-v</code>	<code>cgeist</code> then lift	as <code>lift</code> ; 2 on frontend error
<code>verify</code>	<code>INPUT_FILE</code> ; <code>--sketch</code> ; <code>--z3-timeout</code> (<code>10000</code>); <code>-v</code>	Z3 against one or all 35 sketches; table	0 always (1 on parse error or unknown sketch)
<code>batch</code>	<code>INPUT_DIR</code> ; <code>-o</code> ; <code>-t</code> ; <code>--profile</code> ; <code>--strict</code> ; <code>--z3-timeout</code> (<code>profile</code>); <code>--report</code> ; <code>--report-path</code> ; <code>--include-glob</code> (<code>**/*.mlir</code>); <code>--exclude-glob</code> (<code>corpus/**</code>); <code>--max-files</code> ; <code>--table-limit</code> (<code>100</code>); <code>-write-emitted/--no-write-emitted</code> (<code>write</code>); <code>--top-slowest</code> (<code>10</code>); <code>--validate-emitted</code> ; <code>--mlir-verifier</code> ; <code>-v</code>	Lift every matching file (skips <code>*_lifted.mlir</code>), write artifacts next to sources or to <code>-o</code> , optional validation and JSON report	1 on validation failures; 1 if strict and any unproved or refuted
<code>sketches</code>	<code>-d linalg</code>	stablehlo	all`
<code>demo</code>	<code>-t linalg</code>	stablehlo; <code>--profile</code> ; <code>--z3-timeout</code> (<code>profile</code>); <code>--strict</code> ; <code>-v`</code>	Lift the 22 <code>DEMO_FIXTURES</code> , print MLIR and diagnostics, summary table
<code>replay</code>	<code>REPORT_FILE</code> ; <code>--entry</code> (<code>0</code>); <code>-v</code>	Rebuild loop info from the report's <code>file</code> , simulate counterexample	1 on missing entry, file or sketch; crashes on z3: <code>"-"</code> (F-17)

Internals worth knowing

- `_validate_verify_policy`: rejects `--no-verify` with `--strict` or a strict profile (`UsageError`).
- `_build_docker_compile_script`: the only place that normalises legacy `-1` dimensions to `?`.
- `_collect_batch_mlir_files`: include globs are relative to the input directory; exclusion uses `fnmatch` on relative paths; sorted; `--max-files` applied after exclusion.
- `_build_canonical_stablehlo_summary` and `CANONICAL_STABLEHLO_SKETCHES`: count required StableHLO ops observed, proved, timed out and refuted; convolution variants are collapsed to `stablehlo.convolution`.
- Batch JSON report (`schema_version` 1.1): top-level `generated_at_utc`, `input_dir`, `output_dir`, `target`, `z3_timeout_ms`, `run_metadata` (run id, requested and resolved profile, strictness, env profile, Python version, platform, LoopHole version, fixtures fingerprint, fixture count, Docker image), `selection`, `summary` (counts, acceptance, validation failures), `canonical_stablehlo_summary`, `slowest` (top 25), `results` (per file: paths, sketch, state, acceptance loose/strict, z3, confidence, time, failed implication, disagreement, mismatch summary, notes, bindings, success, error, emitted file, source SHA-256).

Drawbacks

Severity	Issue	Finding
High	Trusted batch accepts the internal verifier; "no verifier" branch dead	F-10
Medium	<code>--no-verify</code> passes timeout 0	F-11
Medium	Options redeclared per command with drifting defaults; <code>--parametric</code> only on <code>lift</code>	F-24
Medium	Two Polygeist paths with different behaviour and default images	F-23, F-24
Medium	<code>demo</code> depends on <code>loophole.tests.fixtures</code>	F-26
Low	Exit code 2 path unreachable with real lifter	F-13
Low	<code>replay</code> crashes on <code>"-"</code>	F-17
Low	Report helpers duplicated in scripts; version hard-coded	F-23, F-25
Low	Module docstring's command list is mis-indented and incomplete	F-25

Tests

`test_cli_batch_report.py` (13), `test_cli_strict_policy.py` (16), `test_cli_error_messages.py` (11). Most use `click.testing.CliRunner` and monkeypatch `Lifter.lift` and the verifier functions.

Change guide

When adding an option shared across commands, create a shared decorator rather than copying the `@click.option` block. When changing report fields, bump `schema_version`, keep old fields for compatibility, and update `scripts/` and Part 4's schema description.

polygeist_frontend.py (cgeist wrapper)

Purpose

Run `cgeist` locally or in Docker and return MLIR text, with structured errors.

Internals

- `build_command(invocation)`: [`cgeist_bin`, `-S`, `-x`, `c|c++`, `-function=NAME` or `--function=*`, `--memref-fullrank`, `-raise-scf-to-affine`, `-std=...`, `-I ...`, `-D...`, extra args, `-o ...`, `sources`].
- `generate_mlir(invocation)`: if `prefer_docker`, try Docker first (`_try_docker_cgeist` returns `None` if disabled or unavailable). Otherwise run locally; on `FileNotFoundError` fall back to Docker; timeouts and non-zero exits raise `PolygeistFrontendError`; output read from stdout or the output file; empty output raises.
- Docker path: `_resolve_docker_image`, `_build_docker_command` (resolve host paths relative to `cwd`, create the output directory, mount the common parent at `/workspace`, rewrite paths, choose `cgeist++` for C++), `_run_docker_cgeist`.

Drawbacks

Severity	Issue	Finding
Medium	Flag assembly duplicated between local and Docker commands (the Docker one omits <code>-x</code>)	F-23
Low	<code>docker info</code> and <code>docker image inspect</code> probes on every call (up to 16 s when Docker is slow)	n/a
Low	No <code>sed/canonicalize</code> normalisation, unlike <code>compile</code>	F-23
Low	Mounting the common parent of paths on different folders can mount a very large directory (for example a drive root)	n/a

Tests

`test_polygeist_frontend.py` (8), all with `subprocess.run` mocked.

mlir_validator.py (artifact validation)

Purpose

Find and run a verifier for emitted MLIR text.

Internals

Covered in Part 2 (verifier discovery). Functions: `find_mlir_verifier`, `_docker_mlir_image`, `_docker_image_available`, `_docker_mlir_verifier_available`, `is_internal_mlir_verifier`, `is_docker_mlir_verifier`, `_validate_mlir_artifact_internal`, `_validate_mlir_artifact_docker`, `validate_mlir_artifact`.

Drawbacks

Severity	Issue	Finding
High	Internal fallback verifier treated as real by the CLI	F-10
High	No per-dialect verifier; StableHLO always fails in Docker	F-09
Medium	Local command path does not catch <code>TimeoutExpired</code> , which aborts a whole batch	n/a
Low	Docstring discovery order is mis-indented and lists Docker before PATH binaries, which surprises users with a local <code>mlir-opt</code> of a different version	n/a
Low	Docker detection duplicated from <code>polygeist_frontend.py</code>	F-23

Tests

Validation functions are exercised through `mlir_verifier_cmd` in `conftest.py` and the three artifact tests; discovery itself has no unit test.

replay_checker.py (counterexample triage, A-14)

Purpose

Help a human understand a NOT_EQUIVALENT result by evaluating which tensor cells the source and the sketch access at the counterexample's IV values.

Internals

- `parse_z3_binding`: integers, rationals and decimals to float.
- `_is_scalar_iv_name`: any identifier (so tensor function names are also attempted and produce parse errors).
- `_eval_float_expr`: tiny recursive evaluator for `+` `-` `*`, unary minus, parentheses.
- `simulate_source`: evaluates each access's index expressions.
- `simulate_sketch`: aligns dimensions like the verifier, evaluates the sketch's maps, labels operands with source tensor names.
- `replay_counterexample`: builds IV values, both access sets, output cells present on only one side, and a verdict sentence including the failed implication.

Drawbacks

Severity	Issue	Finding
Low	Compares accessed index sets, not values; <code>max_abs_diff</code> never computed	F-25
Low	Crashes when there is no output tensor (<code>None in k</code>)	F-17
Low	Duplicates alignment logic from the verifier	F-23

Tests

`test_a14_counterexample_replay.py` (33).

parser_corpus_compat.py (corpus analysis, B-11)

Purpose

Run the extractor over every `.mlir` file below a directory and classify why files are not compatible.

Internals

`analyze_parser_compatibility(root)` runs `AffineExtractor(validate_with_mlir=False)` on each file and `_classify_incompatibilities: no_loop_construct_detected, no_store_detected, parser_error_diagnostics, scf_iter_args_not_modeled` (any `iter_args()`, including the now-supported affine form), `conditional_region_not_modeled, non_for_loop_control_not_modeled`. Results are counts and per-file rows; the script `scripts/analyze_parser_corpus_compat.py` prints JSON and optionally writes JSON and Markdown.

Drawbacks

Severity	Issue
Low	Classification by substring search over raw text, independent of what the parser actually handled
Low	Committed <code>parser_compat_summary.md</code> is stale (old absolute path, removed class name)

tests/fixtures.py (kernel strings)

Purpose

Central collection of MLIR kernels as Python strings, used by tests, `loophole demo` and `examples/run_demo.py`.

Contents

Group	Constants
Matmul	<code>MATMUL_MLIR (4x4)</code> , <code>MATMUL_128_MLIR</code> , <code>MIXED_AFFINE_SCF_MATMUL_MLIR</code> , <code>MIXED_REALWORLD_MATMUL_MLIR (scf step 2)</code> , <code>MATMUL_DYNAMIC_DIMS_MLIR</code> , <code>MATMUL_UNRANKED_MEMREF_MLIR</code> , <code>MATMUL_DYNAMIC_CONFLICTING_ANNOTATIONS_MLIR</code> , <code>MATMUL_ZERO_DIM_STATIC_MLIR</code> , <code>MATMUL_ITER_ARGS_MLIR</code> (real Polygeist form)
Transpose	<code>TRANSPOSE_2D_MLIR</code> , <code>TRANSPOSE_NONSQUARE_MLIR</code> , <code>TRANSPOSE_3D_PERM_120_MLIR</code> , <code>TRANSPOSE_3D_PERM_201_MLIR</code>
Convolution	<code>CONV_1D_MLIR</code> , <code>CONV_1D_STRIDED_DILATED_MLIR</code> , <code>CONV_2D_SIMPLE_MLIR</code> , <code>CONV_2D_1X1_MLIR</code> , <code>CONV_2D_5X5_MLIR</code> , <code>CONV_2D_STRIDED_DILATED_MLIR</code> , <code>CONV_2D_NHWC_MLIR</code> , <code>CONV_2D_STRIDED_DILATED_REORDERED_MLIR</code> , <code>CONV_2D_VAR_TIMES_CONST_INDEX_MLIR</code>
Dot and matvec	<code>DOT_PRODUCT_MLIR</code> , <code>DOT_PRODUCT_16_MLIR</code> , <code>DOT_PRODUCT_SYMBOLIC_N_MLIR</code> , <code>DOT_PRODUCT_SYMBOLIC_ARITH_MLIR</code> , <code>MATVEC_MLIR</code> , <code>MATVEC_TALL_MLIR</code> , <code>MATVEC_WIDE_MLIR</code>
Elementwise	<code>ELEMENTWISE_ADD_MLIR</code> , <code>ELEMENTWISE_SUB_MLIR</code> , <code>ELEMENTWISE_MUL_MLIR</code> , <code>RELU_MLIR</code>
Reductions	<code>REDUCE_SUM_MLIR</code> , <code>REDUCE_SUM_COLWISE_MLIR</code> , <code>REDUCE_MAX_MLIR</code> , <code>MIXED_SCF_AFFINE_REDUCTION_MLIR</code>
Index normalisation	<code>INDEX_VARIATION_EQ_A_MLIR</code> , <code>INDEX_VARIATION_EQ_B_MLIR</code>
Dicts	<code>DEMO_FIXTURES</code> (22 entries), <code>ALL_FIXTURES</code> (37 entries; omits <code>DOT_PRODUCT_SYMBOLIC_N_MLIR</code> and <code>MATMUL_ITER_ARGS_MLIR</code>)

Drawbacks

Severity	Issue	Finding
Medium	Only correct kernels; no mutants	T1
Medium	Lives in the runtime package	F-26
Medium	<code>MIXED_REALWORLD_MATMUL_MLIR</code> has a step-2 loop and is expected to lift as full matmul	F-03
Low	<code>ALL_FIXTURES</code> mixes two indentation styles and silently overwrites duplicate keys	n/a
Low	The same kernels also exist as <code>.mlir</code> files under <code>tests/fixtures/</code> with slightly different names, so there are two fixture sources	n/a

Scripts and Examples

scripts/generate_weekly_benchmark_report.py

Runs `Lifter(target, z3_timeout_ms)` over every non-lifted `.mlir` below `--fixtures-dir` (default `tests/fixtures`, including `corpus/`), and writes `weekly_benchmark_<label>_<timestamp>.json` and `.md` to `--output-dir` (default `packages/loophole/reports/benchmarks`, git-ignored). Report `schema_version` 1.2 adds `trend_summary` (deltas against `--baseline-report`, anomalies for proved drop, refuted increase and latency regression above `--regression-threshold-fraction`), `canonical_stablehlo_summary` and `proof_quality_summary` (proof rate, strict acceptance rate, confidence distribution, per-sketch breakdown, corpus vs unexpected refuted, grade A to F). No profile, no strict mode, no artifact validation. See F-28.

scripts/generate_proof_quality_summary.py

Standalone A-16 artifact (`proof_quality_<label>_<timestamp>.json/.md`) that imports underscore helpers from the weekly script by manipulating `sys.path`. Same caveats.

scripts/analyze_parser_corpus_compat.py

Wrapper around `parser_corpus_compat` with `--corpus-dir`, `--json-out`, `--md-out`. Unlike the other scripts it does not add `src` to `sys.path`, so it needs the package installed.

scripts/run_batch_report_in_docker.ps1 and run_weekly_report_in_docker.ps1

PowerShell only. Mount the repository root at `/workspace/src`, create a virtual environment inside the container, `pip install` requirements and the package (network needed every run), then run `loophole batch --report` or the weekly script. The image default is the GHCR name.

examples/

File	Purpose
<code>smoke_matmul.c</code>	2x2 matmul with a local accumulator: produces the <code>iter_args</code> form; the smoke test in <code>SETUP.md</code>
<code>matmul_inplace.c</code> , <code>matmul_inplace_128.c</code>	In-place <code>C[i][j] = C[i][j] + ...</code> form at 4x4 and 128x128, added for the GPU full-workflow run
<code>run_demo.py</code>	Rich or plain demo over <code>DEMO_FIXTURES</code> with <code>target="both"</code> , plus a step-by-step matmul walkthrough; smoke-tested by <code>test_demo_script_smoke.py</code>